

NMEA2000 display with autopilot and alarm handling:

Based on the work of

- [Hombberger](#)
- [Timo Lappalainen](#)

What to expect ?

- A fully working NMEA2000 display comparable to some commercial displays.
- No soldering needed: just connect the NMEA2000 wires and the supply wires (4 wires) to the terminal block
- The 3D printable housing has the same mounting holes as an ST70 instrument.
- Compatible with Raymarine EVO autopilot and different autopilot modes.
- Alarm display for the autopilot related alarms and ability to acknowledge the alarms
- Customization to different Units (knots, km/h, m/s, ft, m, ...) with a settings screen. Settings are stored in flash

What is NOT to be expected ?

- The display has not enough brightness to be used outdoors (350 Nits)
- The display is not rugged enough to be used outdoors
- The housing is not watertight and not designed to be used outdoors

I am trying to negotiate with waveshare to make a high NIT board available. At this point in time they are not interested because I can not promise firm quantities. If you want to help me (and yourselves) in this negotiation, send a mail to waveshare asking for a high NIT versions of their board (ESP32-S3-Touch-LCD-4) by submitting a ticket on their website (https://www.waveshare.com/wiki/ESP32-S3-Touch-LCD-4?srltid=AfmBOopnndw3P_ItxOh3U-3SY4NCcHN8WxyNpG35UeajusQ1Xy2rScYm#Support)

What hardware is needed for this project ?

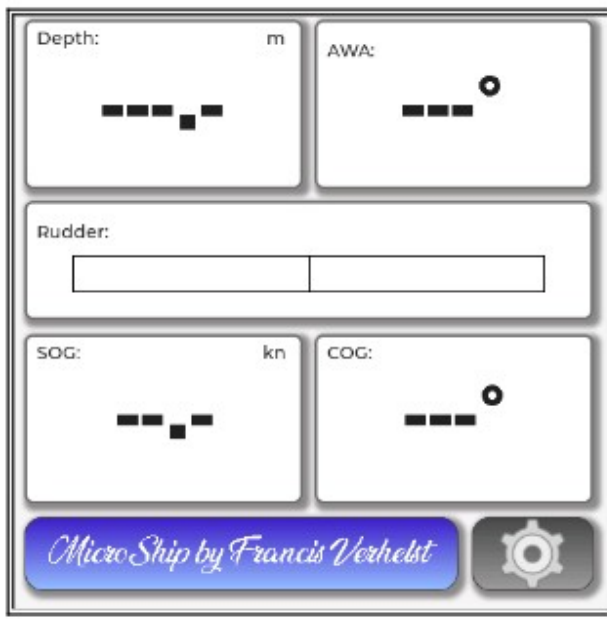
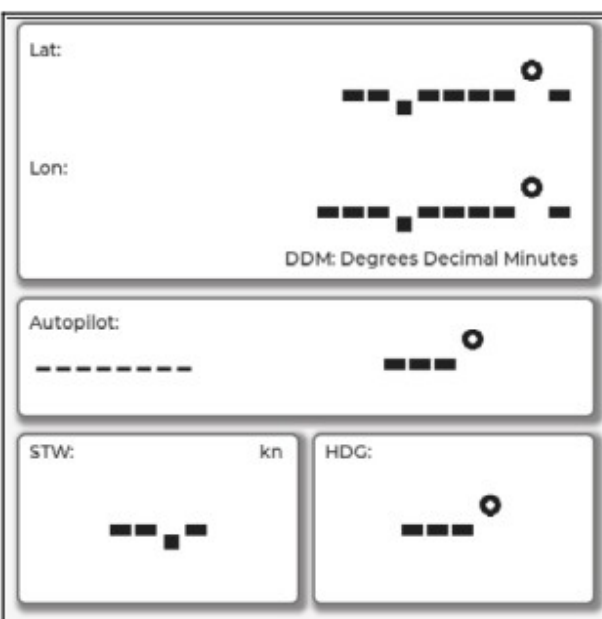
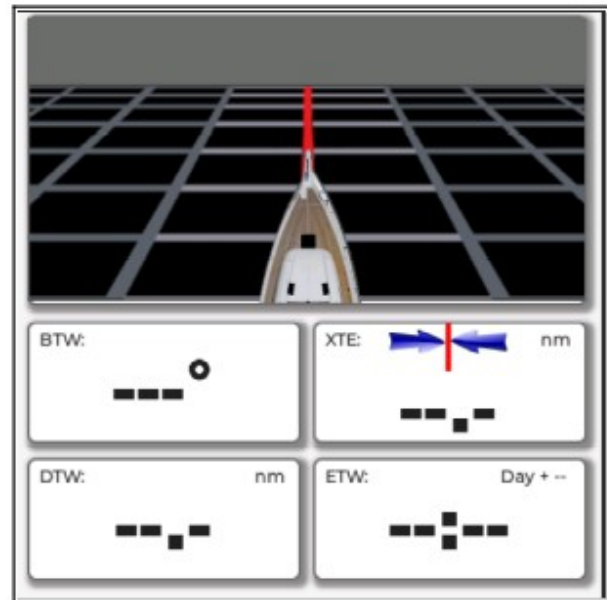
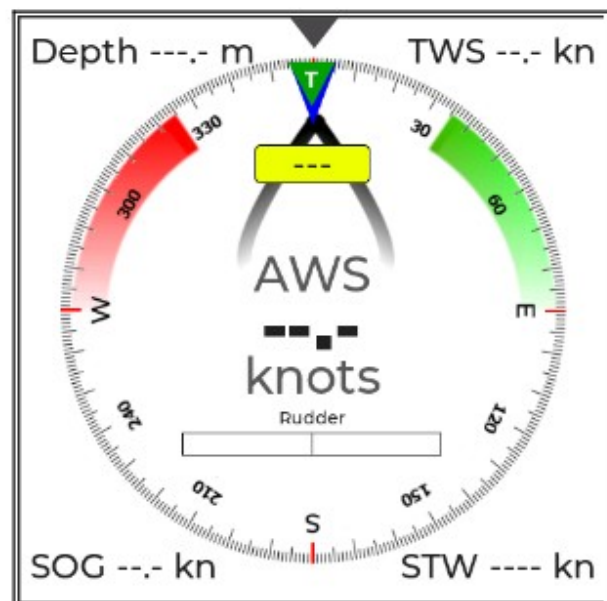
- A waveshare ESP32-S3-Touch-LCD-4 Version 4 board. I have provided a flash file provided for V3 boards, but not the source files. I can provide the V3 source files, but some functionality is not available on V3 boards: No backlight control and power up of the board is not automatic but has to be done with the power key. Please send me a message if you need the Version 3 source files.
- An NMEA2000 cable that is compatible with your boat network (in my case a Raymarine SPUR cable)

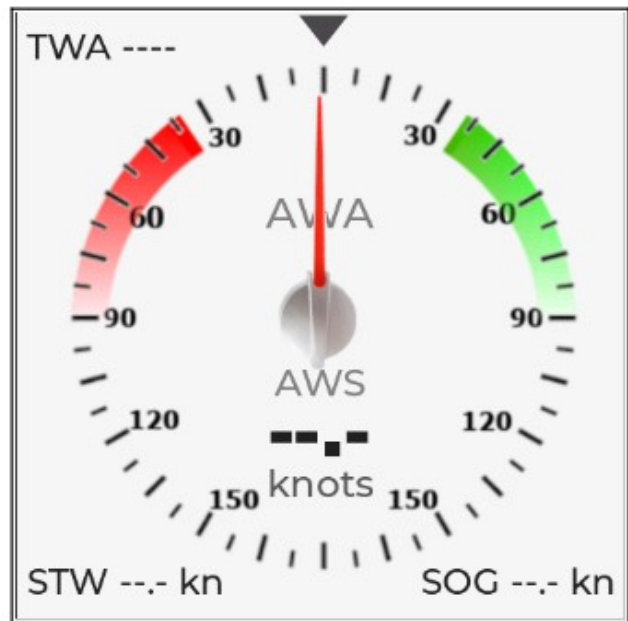
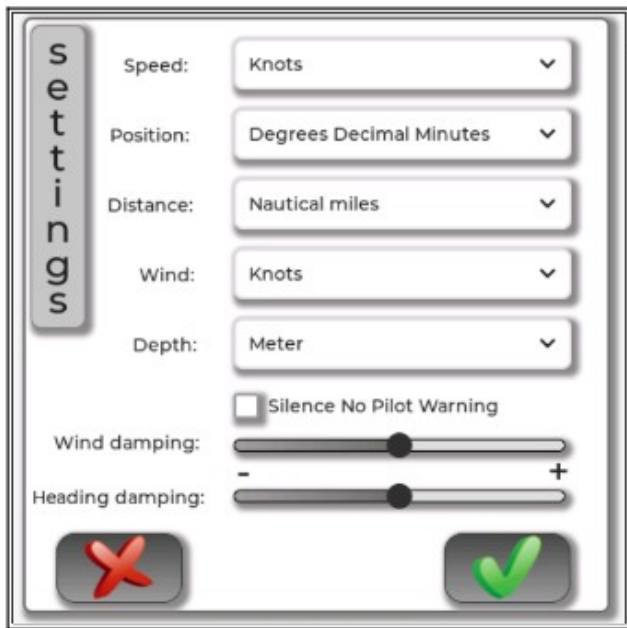
Like this project? Even a tiny \$1 or \$2 donation via Sponsor helps me keep working on it. On the right hand side of the web page you find a



buymeacoffee.com/francissailor button.

Some screenshots:





(IMAGES ARE SCREENSHOTS, THUS NOT ANIMATED WITH REAL VALUES from NMEA2000)

Introduction:

I had a Raymarine ST70 display which was not functioning correctly anymore. Buying a new one ? Between 500 and 600 Euro... This is when I started looking for an alternative. In the end I came to the conclusion that the only affordable solution was to program it myself. The main objectives:

- affordable !
- a User Interface comparable or better than the old Raymarine
- no custom PCB design and if possible without need to solder anything

Hardware:

After searching through internet, I found the Waveshare ESP32-S3-Touch-LCD4 board with integrated can bus on board, supply voltage up to 37 Volt and a beautiful 480x480 LCD touch screen. The price is about 35 Euro and indeed no soldering at all is needed. Beware: there are 2 versions of this board ! Check to see if the one you want to buy has the integrated can bus. This board is delivered without housing, so I designed a housing in freecad myself.

Software development:

I developed the program / sketch with the Arduino IDE. For the UI I used Squareline studio. For the NMEA200 library I used the well known library of Timo Lappalainen (<https://github.com/ttlappalainen>) After 6 months of programming (and with a little help from chat GTP for some of the routines) I am proud to share the finalized project. I also included a link to give me a small donation if you appreciate my work. (my wife hopes I will get rich with my programs :))

Software peculiarities:

The software uses in my opinion the full capacity of the ESP32 chip:

- the NMEA2000 library and decoding logic runs on a RTOS task on core 0. The use of core 0 was necessary because NMEA2000 has a lot of network traffic at high speeds. With the classic approach of writing one huge sketch running on core 1 I missed part of the NMEA2000 messages.
- the user interface uses the FATFS partition of 9,9MB to store all graphical data. On startup this data is copied over to PSRAM. The reason for this approach is that the normal flash partition for the sketch is too small to contain the graphics data. Furthermore I ran in to problems with the amount of

normal RAM. Using the FFAT partition and PS RAM was the only solution to this (except for an SD card, a solution I did not want for reliability of the display). Why copy all this data to PSRAM ? PSRAM, although slower than the main RAM, is still much faster than flash memory, which is an important factor in screen refreshes. The use of FFATFS means you will need to write the graphical output data to FATFS with a small FFAT up loader sketch. This auxiliary program is added in the download files. Alternatively, you can use ESPConnect

(<https://github.com/thelastoutpostworkshop/ESPConnect>) More details on how to proceed in sections below.

- There is a small auxiliary task that only is used for the onboard beeper. When I integrated the beep routine in the sketch for the user interface based on millis() the beep interval was irregular due the burden of screen updates on the ESP32. This small RTOS task also runs on core 0.

for the tinkerers

who want to change or adapt my software: the LCD display of this waveshare board is RGB. This means that timings from the microcontroller to the LCD are critical. I fine tuned the timings in the library lvgl_port_v8.h. Changing these settings may corrupt the display ! This is a trial and error approach...

Please also pay attention to the library versions as indicated on the waveshare wiki

(<https://www.waveshare.com/wiki/ESP32-S3-Touch-LCD-4>)

This also has implications for the version of Squareline studio you need, since the last version dropped support for LVGL 8.4

AND BEWARE: MODIFYING OR ALTERING THIS PROGRAM IS NOT FOR BEGINNERS !!

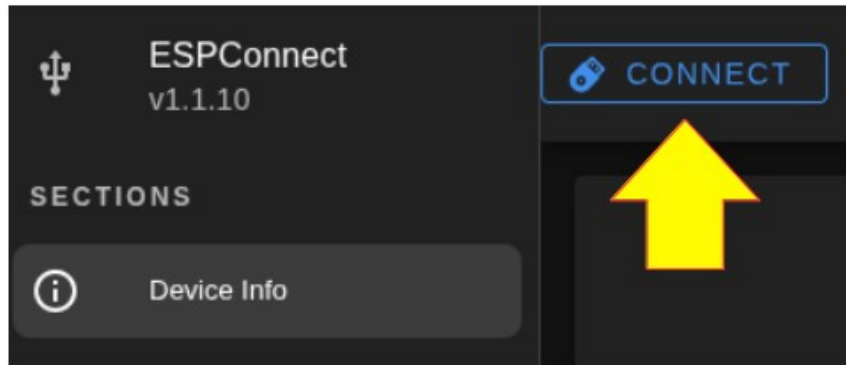
OKAY, now hands on: How do you implement this project ?

The easy method:

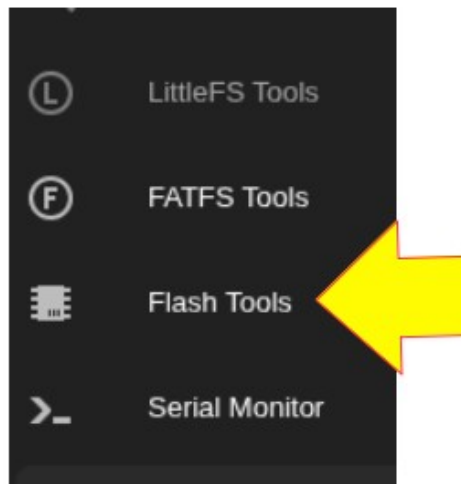
in the files section there is a directory “bin file”. This bin file has been generated by ESP connect (<https://thelastoutpostworkshop.github.io/ESPConnect/>) and can be uploaded directly to the ESP32-S3 board. Choose the file depending on the version of your board ! (version 3 or 4)

In ESP connect:

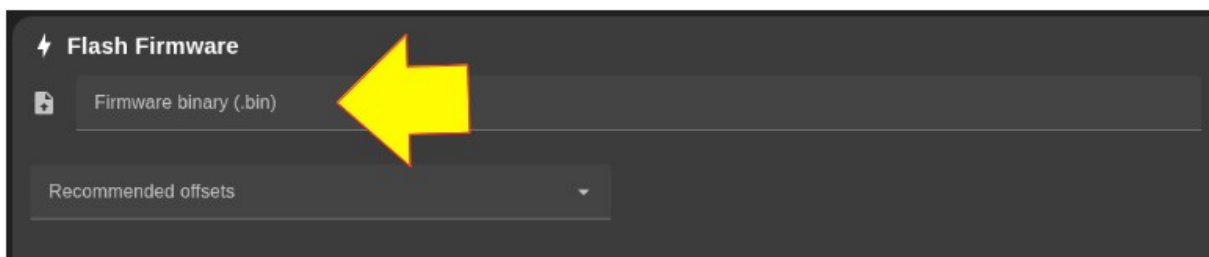
- first you need to connect to the ESP32-S3 by choosing the correct USB port



- Then you need to click on Flash Tools



- Here you need the section Flash Firmware. Click on the Firmware binary (.bin) and direct it to the bin file which in the repo under directory bin file



- Now click on Flash Firmware



That's it, your board now has the full program. You only need to connect it to your boat network

For the tinkerers:

The source files and the used libraries are in the respective folders.
BUT: please follow these steps:

1. Load assets (image data) to FFAT:

First we have to load all graphical files that are in the “FFAT drive” under the “squareline studio” folder to the FFAT partition of the Waveshare board. For this we use the little helper program in the “FFATuploader” folder. Compile and upload this sketch to your board.

You now have to connect to the wifi from the ESP32-S3 access point "ESP32S3_FFAT_UPLOAD" with password 12345678

Open a browser window on address 192.168.4.1. You will now be presented with a simple web interface and fill in the field for “Doelmap” (target directory in FFAT) with /assets.

Now choose the files to upload by clicking on “Browse...”. A window will open to allow you to choose the files to upload. Go to the directory mentioned above (“FFAT drive” under the “squareline studio” folder). Select all files and press the open button on the bottom of the window.

Finally click on the button “Upload”. The upload to the ESP32-S3 will now start. Please be patient ! This takes about 20 seconds to complete.

ESP32-S3 FFat Uploader

Verbonden met: ESP32S3_FFAT_UPLOAD

Selecteer één of meerdere bestanden en kies een doelmap.

Doelmap:

Voorbeelden: /, /assets, /audio/sets.
De map moet bestaan of wordt (indien mogelijk) aangemaakt.

No file

Mappen beheren

Nieuwe mapnaam:

Geen ... gebruiken. Geneste paden zijn toegestaan.

FFat beheer

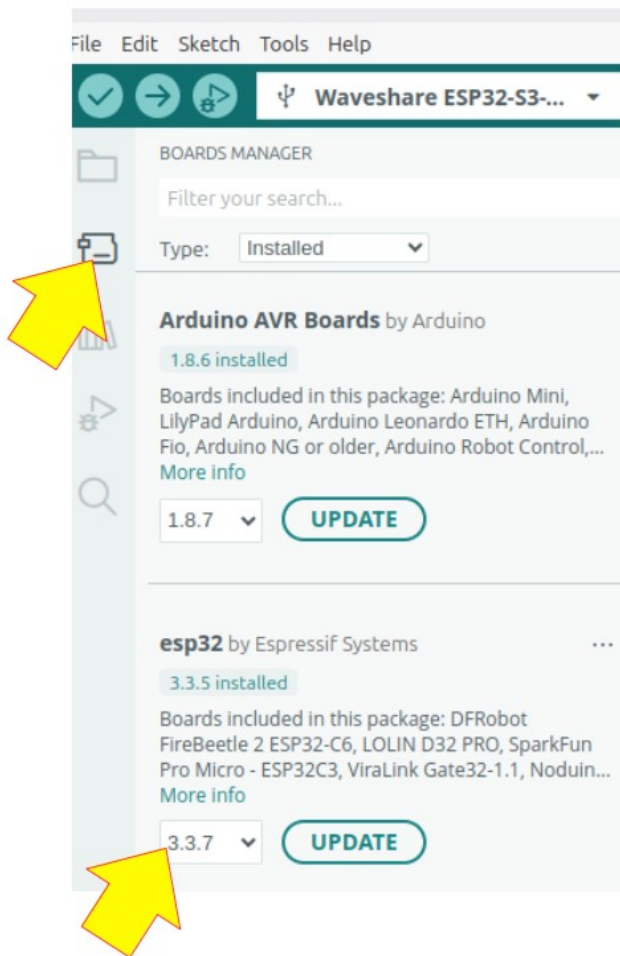
Name	Size	Type
ui_img_1883567225.bin	4,2 kB	Binary
ui_img_appwind_png.bin	57,0 kB	Binary
ui_img_boat_png.bin	125,1 kB	Binary
ui_img_brightness_png.bin	4,7 kB	Binary
ui_img_cancel_png.bin	8,1 kB	Binary
ui_img_circularscale_png.bin	366,4 kB	Binary
ui_img_compass2_png.bin	534,3 kB	Binary
ui_img_contour_png.bin	297,4 kB	Binary
ui_img_error2_png.bin	30,0 kB	Binary
ui_img_error_png.bin	26,7 kB	Binary
ui_img_helmmann_png.bin	221,8 kB	Text
ui_img_indicator_png.bin	3,5 kB	Binary
ui_img_needle_png.bin	44,2 kB	Binary
ui_img_ok_png.bin	8,1 kB	Binary
ui_img_settings_png.bin	10,8 kB	Binary
ui_img_splash_png.bin	691,2 kB	Text
ui_img_truewind_png.bin	55,7 kB	Binary
ui_img_turn_port_png.bin	29,7 kB	Binary
ui_img_turn_starboard_png.bin	29,7 kB	Binary
ui_img_warning_png.bin	27,9 kB	Binary
ui_img_windcontour_png.bin	760,5 kB	Binary

Ok, The FFAT is ready now.

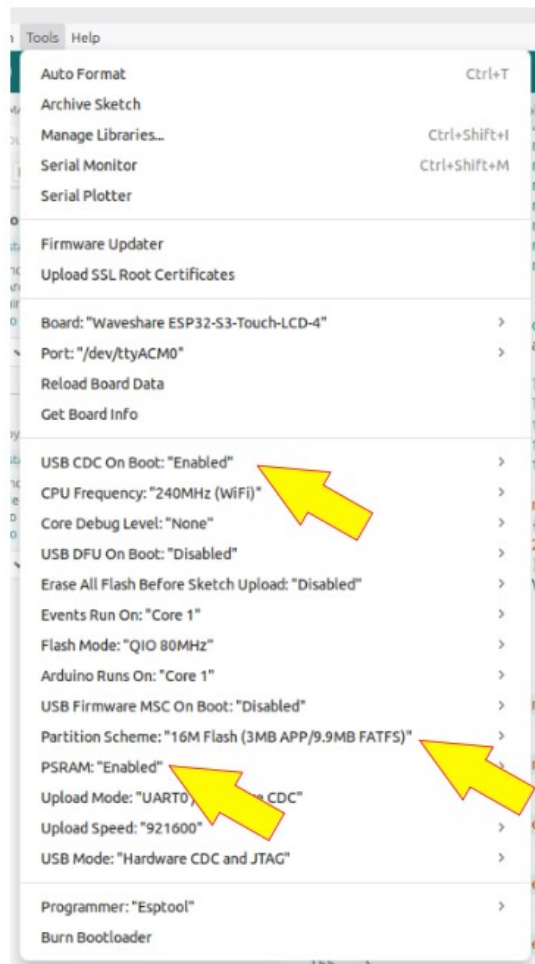
2. Compile and upload the sketch in the Arduino IDE:

This step is straight forward. Only pay attention to:

- Choose the correct board in the Arduino IDE. In the board manager I have the board manager for ESP32 version 3.3.7 installed



- Choose the correct settings for the waveshare board



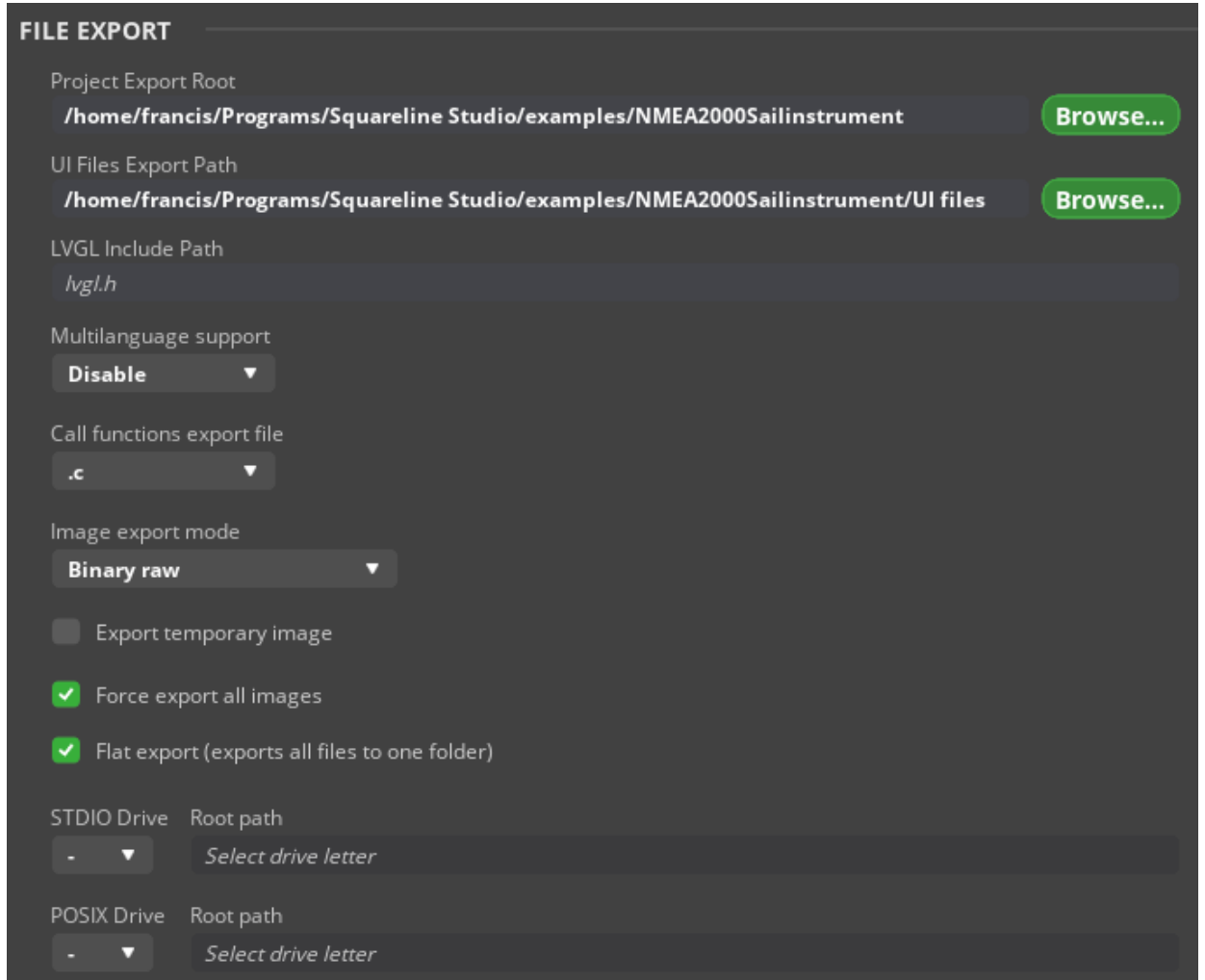
Now compiling and uploading should just work fine.

As a side note: I started this project under Windows 10 and the compile speed of the Arduino IDE is painfully slow. After my migration to Linux Mint (in dual boot) I notice that compile speed that is much much faster.

Squareline project:

The full Squareline project is in the Squareline folder. If you modify the Squareline project there are some important rules to respect:

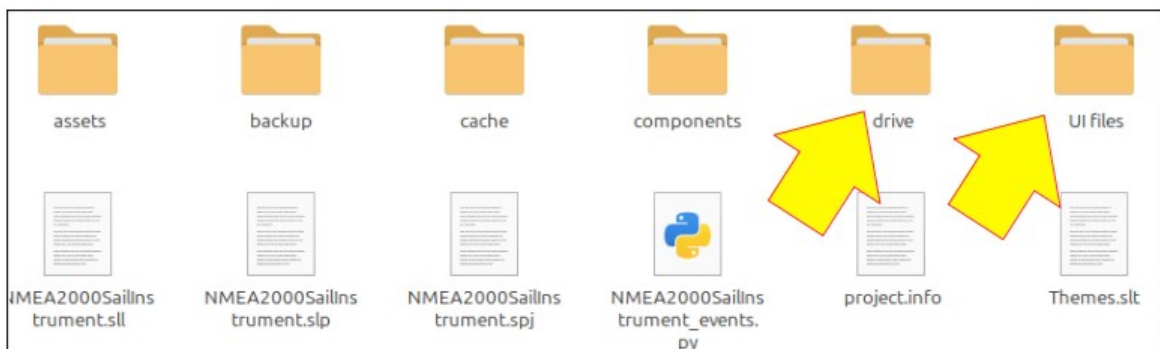
- Use Squareline studio version 1.5.4. The newer versions drop support for LVGL8.4. The waveshare board runs on LVGL versions up to 8.4. Later versions of the LVGL library will not work with this board !
- The project settings need to be set on:



The screenshot shows the 'FILE EXPORT' settings window in Squareline Studio. The settings are as follows:

- Project Export Root:** /home/francis/Programs/Squareline Studio/examples/NMEA2000Sailinstrument (with a 'Browse...' button)
- UI Files Export Path:** /home/francis/Programs/Squareline Studio/examples/NMEA2000Sailinstrument/UI files (with a 'Browse...' button)
- LVGL Include Path:** lvgl.h
- Multilanguage support:** Disable (dropdown menu)
- Call functions export file:** .c (dropdown menu)
- Image export mode:** Binary raw (dropdown menu)
- Export temporary image:** ☐
- Force export all images:** ☒
- Flat export (exports all files to one folder):** ☒
- STDIO Drive Root path:** - (dropdown menu) with a hint 'Select drive letter'
- POSIX Drive Root path:** - (dropdown menu) with a hint 'Select drive letter'

- The Squareline studio export UI files will now export to different directories. 2 folders are important: the drive folder and the ui files folder



- All files in the UI files folder need to be copied to the folder of the Arduino sketch. EXCEPT 2 FILES: DO NOT COPY UI_IMG_MANAGER.C AND UI_IMG_MANAGER.H The correct versions of these 2 files are already in the Arduino sketch folder. I modified these files to take into account that they are in PS RAM.

ui_img_error2_png.c	532 bytes	Text
ui_img_error_png.c	525 bytes	Text
ui_img_helmmann_png.c	534 bytes	Text
ui_img_indicator_png.c	546 bytes	Text
ui_img_manager.c	523 bytes	Text
ui_img_manager.h	168 bytes	Text
ui_img_needle_png.c	531 bytes	Text
ui_img_ok_png.c	504 bytes	Text
ui_img_settings_png.c	542 bytes	Text
ui_img_splash_png.c	534 bytes	Text

- If you added or replaced any assets, all assets will be in the drive folder. You will need to upload them to the FFAT partition as described above in **1. Load assets (image data) to FFAT:** (I reformat the partition before re uploading the assets to FFAT. In the FFAT up loader tool a button is provided for this)

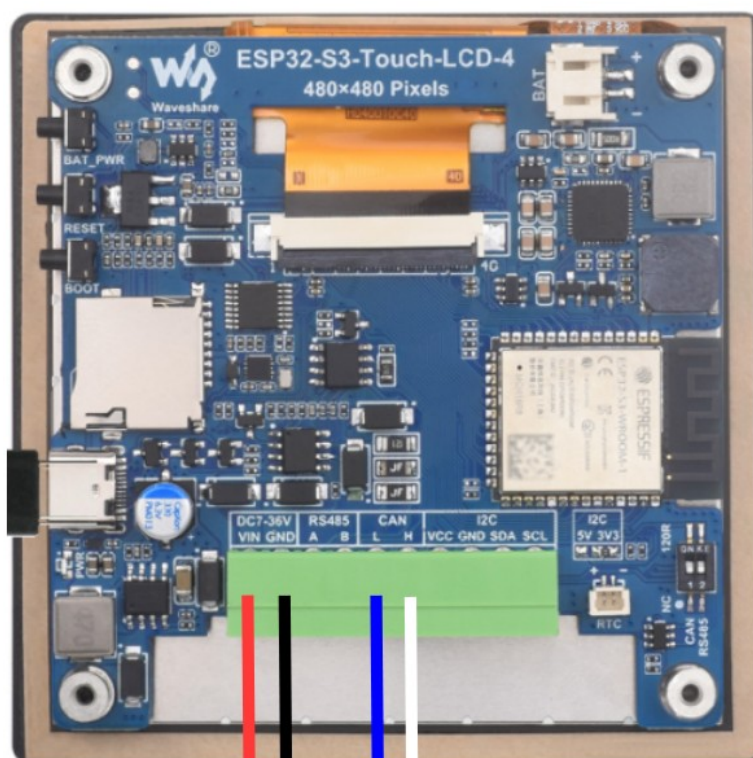
Housing:

There is also a folder for the housing, with the FreeCAD files. The housing was designed with FreeCAD 1.0, but the files are compatible with version 1.1. Files you can use directly for your 3D printer are also included in this folder.

There are mounting holes for mounting the display with small 2,5 mm screws, but i noticed it is very easy to crack the glass of the LCD if you tension even only a little bit too much the screws. I now glue the display in place, which is safer.

Hardware Connections:

easy: use a NMEA2000 cable and cut it to the desired length. For Raymarine Seatalk NG the colours are: Red to Vin, Black to GND, white to CAN H, blue to CAN L. This is all.



If you have problems or if more clarification needed, please send me a mail
franciscontact@hotmail.com